VERIQTA

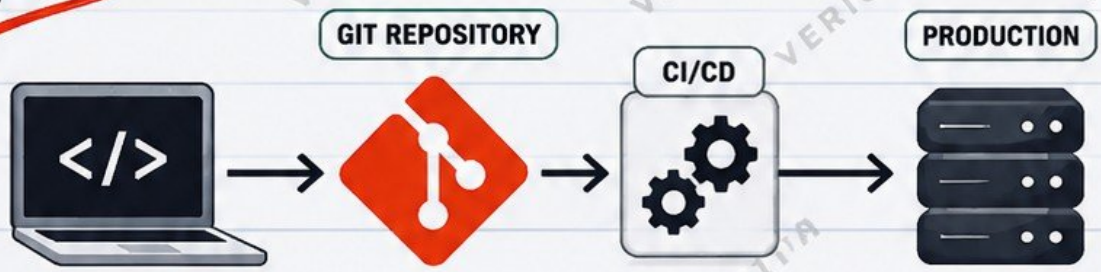
LEARN | PRACTICE | DEPLOY

Git IN PRODUCTION

A PRACTICAL JUNIOR ENGINEER HANDBOOK

From Commits to Production

Real Workflows Real Scenarios Real Skills



- Real Workflows
- Branching Strategies
- Pull Requests
- CI/CD Integration
- Infrastructure as Code
- Releases and Rollbacks
- Security Best Practices
- Incident Troubleshooting
- Real-World Examples
- Hands-On Labs

Build Deploy Operate with Git

TURN VERSION CONTROL INTO REAL-WORLD IMPACT

Same Engineers Different Results

Code | Automate | Deploy | Scale

JUNIOR ENGINEER TO PRODUCTION

VERIQTA

What "Git in Production" Actually Means

Understand how Git fits into real-world production environments.

1. Git Itself vs Production Environment

- Git is a version control system. It stores changes to files over time.
- Git does not run your application.
- Git is not a server, container, cloud, or production environment.
- Production is where your application runs and is accessed by users.
- Git and production are connected, but they are different things.

2. Git as the Source of Truth

- Git stores the source code for applications.
- It also stores infrastructure as code, Kubernetes manifests, CI/CD configurations and documentation.
- Teams use Git as the single source of truth for what should be deployed.
- Every change is tracked with who, what, when and why.
- This history helps with auditing, rollbacks and troubleshooting.

3. Why git push Is Not Deployment

- git push only sends your changes to a remote repository (for example GitHub).
- It does not build, test, package or deploy your application.
- Your changes are just stored in the repository.
- A CI/CD pipeline must take those changes, build an artifact and deploy to the production environment.
- Never assume code in a repository is already running in production.

4. Application vs Infrastructure Repositories

Application Repository

- Application source code
- Configuration files
- Dependencies
- Tests
- CI/CD pipeline files
- Documentation

Infrastructure Repository

- Infrastructure as code (Terraform, CloudFormation)
- Ansible playbooks
- Kubernetes manifests
- Environment configurations
- CI/CD for infrastructure
- Runbooks and documentation

5. Production Awareness

- Git records the desired state (what should be deployed).
- Production systems run the deployed version (what is actually running).
- There may be commits in Git that are not yet deployed.
- There may be running code in production that was deployed from a specific version, tag or commit.
- Always verify what version is running in production.

Remember This

Git records desired changes.
Production systems run deployed versions.

6. The Typical Flow From Code to Production

Developer — Writes and commits changes →

Git Repository — Stores code, history and configuration →

CI/CD — Builds, tests, scans and packages the code →

Artifact — Creates a versioned artifact (for example container image) →

Deployment — Deploys to the target environment →

Production — Application runs and is accessed by users

Junior Engineer Tip

Learn the full flow.
Git is the beginning, not the end.

The Production Git Repository

A well-structured repository is the foundation of reliable production systems.

1. Repository Anatomy

- A production repository contains more than just application code.
- It holds everything needed to build, deploy and operate the system.
- It is the single source of truth for the team.
- It enables collaboration, review, audit and automation.
- A well-organized repository makes production systems more reliable.

2. Typical Production Repository Structure

my-project/	
app/	Application code
infrastructure/	Infrastructure as Code (Terraform, CloudFormation)
kubernetes/	Kubernetes manifests
.github/	CI/CD configuration
docs/	Documentation and runbooks
README.md	Project overview and setup
.gitignore	Files to exclude from Git
CODEOWNERS	Defines repository owners
LICENSE	License information

3. Key Components in a Production Repo

- **Application Code**
  Source code for the application or service.
- **Infrastructure as Code**
  Terraform, CloudFormation, Ansible, etc.
- **Kubernetes Manifests**
  Deployment, Service, Ingress, ConfigMaps, etc.
- **CI/CD Configuration**
  GitHub Actions, GitLab CI, Jenkins files, etc.
- **Documentation**
  Runbooks, architecture diagrams, operations guides.

4. Important Files

README.md	Explains the project, setup and how to use it.
.gitignore	Specifies files and folders that should not be tracked.
CODEOWNERS	Defines who is responsible for code changes.

5. Repository Ownership

- Every repository should have clear owners.
- Use CODEOWNERS to define responsible teams.
- Set review requirements for sensitive paths (like infrastructure or Kubernetes).
- Ensure changes are reviewed by the right people.
- Keep access permissions up to date.

6. What Should Never Enter Git

- Passwords and credentials
- API keys and tokens
- SSH private keys
- Cloud credentials (AWS, Azure, GCP, etc.)
- .env files with sensitive data
- Database connection strings
- Personal or financial information
- Large binaries or sensitive log files

Security Reminder

A private repository is not a secrets manager.

Git is for storing code and configuration, not sensitive information. Use a proper secrets management solution (for example, AWS Secrets Manager, HashiCorp Vault, Azure Key Vault).

Junior Engineer Tip

Keep your repository clean, organized and secure. It will save you and your team from production incidents.

Production Branching Strategies

Choose the right branching strategy for reliable delivery.

1. Why Teams Use Branches

- Branches allow multiple people to work without breaking each other's changes.
- They isolate new features, fixes and experiments.
- Teams can review, test and integrate changes safely.
- Branches enable better collaboration, auditability and rollback options.

2. Short-Lived Feature Branches

- Used for a single feature, bug fix or small change.
- Created from main (or develop).
- Kept for a short time (days or weeks).
- Merged after review and tests pass.
- Reduces merge conflicts and keeps the repository clean.

```
git checkout -b feature/login
git push -u origin feature/login
```

3. Trunk-Based Development

- Develop directly on main (trunk) with small, frequent changes.
- Uses feature flags to hide incomplete work.
- Requires automated tests and strong CI/CD.
- Reduces long-lived branches and merge conflicts.
- Popular in modern DevOps and cloud-native teams.

4. GitHub Flow

- Simple and widely used.
- Feature branches from main.
- Pull request for review.
- Merge to main after checks pass.
- Works well for continuous deployment teams.
- Ideal for small to medium teams.

5. GitFlow

- Uses multiple long-lived branches (main, develop, feature, release, hotfix).
- Common in larger or traditional teams.
- Supports scheduled releases.
- More complex to manage.
- May still appear in some enterprises with strict release cycles.

6. Long-Lived Branches: Risks

- Harder to merge as they diverge from main.
- More conflicts and integration work.
- Changes may become outdated.
- Higher operational and maintenance cost.
- Can slow down delivery.
- Use only when necessary.

7. main as a Protected Branch

- main should always be stable and production-ready.
- Use branch protection rules (required reviews, status checks).
- Prevent direct pushes, force pushes and deletion.
- Only merge through pull requests.
- This ensures higher quality and safer deployments.

8. Choosing a Strategy

- Small teams and continuous deployment → GitHub Flow.
- Modern cloud-native teams → Trunk-Based.
- Large teams with scheduled releases → GitFlow.
- Consider your team size, release frequency, compliance requirements and deployment model.

9. Comparison: Trunk-Based vs GitHub Flow vs GitFlow

Feature	Trunk-Based	GitHub Flow	GitFlow
Main Idea	Develop on main, frequent changes	Feature branches, PR to main	Multiple long-lived branches
Branches	Short-lived (or none)	Short-lived feature branches	Feature, develop, release, hotfix
Complexity	Low	Low	High
Best For	Cloud-native, DevOps, fast delivery	Small to medium teams	Large teams, scheduled releases
Release Model	Continuous deployment	Continuous deployment	Scheduled releases
Uses Feature Flags	Yes (recommended)	Optional	Sometimes
Operational Cost	Low	Low	High
Popularity	Growing (modern teams)	Very popular	Still used in some enterprises

Junior Engineer Tip

Use the simplest strategy that meets your team's needs.

A Production Change From Start to Finish

Follow one realistic change through its complete lifecycle.

1. **Issue** — A problem, feature request or task is created in the issue tracker (for example, GitHub Issues, Jira, or Azure Boards).
2. **Branch** — Create a new branch from main (for example, feature/fix-login).
3. **Code Change** — Make the required code or configuration changes in your local environment.
4. **Commit** — Save the changes with a clear and meaningful commit message.
5. **Push** — Push the branch to the remote repository (for example, GitHub).
6. **Pull Request** — Open a pull request to propose the change and start the review process.
7. **Automated Checks** — CI runs tests, builds, security scans and other automated checks.
8. **Review** — Team members review the code, discuss and approve the change.
9. **Merge** — The approved pull request is merged into main (or the target branch).
10. **Build** — CI builds the application or infrastructure and creates an artifact (for example, a container image or package).
11. **Deploy** — CD deploys the artifact to the target environment (staging or production).
12. **Verify** — Verify the deployment by checking application health, logs, metrics and functionality.

Where Git Stops and CI/CD Begins

- Git handles source code, branches, commits, pull requests and merge history.
- Git records what should be deployed (the desired changes).
- CI/CD takes those changes, builds, tests, creates artifacts and deploys to environments.
- Git does not deploy your application. Pushing code is not a deployment.
- Deployment happens through an automated pipeline or a controlled process.

Why Verification Matters

- Confirms the change is working as expected.
- Detects issues early before users are impacted.
- Ensures the right version is running in production.
- Allows quick rollback if something is wrong.
- Builds confidence in the release process.
- Verification should include application checks, logs, metrics and real user functionality.

Production Awareness

- Git records the desired state (your changes).
- Production systems run the deployed version (artifact), not your latest commit.
- Always verify what is actually running in production.
- Not all commits are deployed.
- Use tags, release versions or commit SHA to track what is in production.

Junior Engineer Tip

A successful change is not complete until you verify it in production.

Remember

git push is not a deployment.

Common Mistake

Assuming the code is in production after merging. Always verify.

Writing Production-Quality Commits

Clear commits make your team, your code and your production systems more reliable.

1. What a Commit Actually Represents

- A commit records a snapshot of your changes at a point in time.
- It stores the content, author, message and timestamp.
- It becomes part of the project history.
- Commits help teams understand what changed, why and when.

2. Atomic Commits

- Each commit should make one logical change.
- Keep commits small and focused.
- Easier to review, test and rollback.
- Avoid mixing unrelated changes (for example, a feature and code cleanup).

One commit = one clear purpose.

3. Good Commit Boundaries

- A single feature or bug fix.
- A specific configuration change.
- A small refactor.
- A documentation update.
- Do not combine multiple features in one commit.
- Keep changes easy to understand, review and revert if needed.

4. Useful Commit Messages

- Clear, concise and descriptive.
- Explain what changed and why.
- Use present tense (not past tense).
- Mention the component or area.
- Keep the first line short (50–72 characters).
- Add more details in the description if needed.

5. Conventional Commits (Optional)

- A common commit message standard.
- Helps with automation, changelogs and releases.
- Format:

```
type(scope): short description
```

- Examples:

```
feat(api): add user login endpoint
fix(k8s): correct liveness probe
docs(readme): update setup guide
chore(deps): update dependencies
```

6. Poor Commit Message Example

✕ **fixed stuff**

- Too vague and not descriptive.
- Does not explain what changed.
- Makes troubleshooting difficult.
- Unhelpful for code reviews.
- Avoid generic messages like:
  - fixed stuff
  - update
  - changes
  - misc
  - final version

7. Essential Git Commands for Committing

Check the status

```
git status
```

- Shows the current state of your working directory.
- Tells you which files are modified, staged or untracked.

Review the changes

```
git diff
```

- Shows exactly what changes you have made.
- Review your diff before adding or committing.
- Helps you catch mistakes.

Stage the changes

```
git add <file>
git add .
```

- Adds changes to the staging area.
- Use a specific file or a dot (.) to add all changes.

Create the commit

```
git commit -m "your message"
```

- Creates a commit with your staged changes.
- Use a clear and meaningful message.

8. Reviewing Before You Commit

- Always run git status and git diff.
- Check that you are only committing what you intended.
- Look for sensitive information (credentials, keys, tokens).
- Remove unnecessary files.
- Make sure your commit message is clear.

Junior Engineer Tip

Inspect the diff before every production-related commit.

Take a few seconds to review your changes. It can save you from production issues later.

10. Production Impact

- Clear commits make code reviews faster and safer.
- Good history helps with debugging and incident response.
- Small, focused commits are easier to rollback.
- Your future self and your team will thank you.

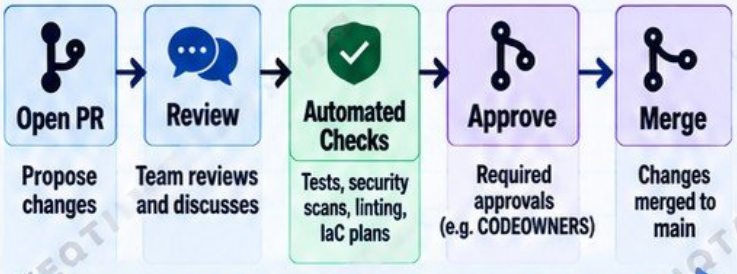
Pull Requests as a Production Safety Gate

Review. Validate. Approve. Deploy with confidence.

1. Why Production Teams Review Changes

- Catches mistakes before they reach production.
- Improves code quality and maintainability.
- Brings different perspectives and expertise.
- Ensures security and compliance requirements.
- Helps with knowledge sharing and team alignment.
- Reduces the risk of outages, incidents and rollbacks.
- Creates an audit trail of who changed what and why.

2. Pull Request Lifecycle



3. Small vs Massive PRs

Small PRs

- Easier to review
- Faster feedback
- Less chance of conflicts
- Focused on one change
- Higher quality
- Quicker to merge

Massive PRs

- Harder to review
- Slower feedback
- More merge conflicts
- Multiple unrelated changes
- Higher risk of bugs
- Often gets delayed

Keep PRs small, focused and easy to review.

4. What Gets Checked in a PR

- Peer review (human review)
- Automated tests (unit, integration)
- Security scans (SAST, DAST, dependency checks)
- Infrastructure plan reviews (for example, terraform plan)
- Code style and linting checks
- Compliance and policy checks
- Required approvals
- Documentation updates (if needed)

5. CODEOWNERS

- A file called CODEOWNERS defines who must review specific files or directories.
- Useful for sensitive areas like infrastructure, security and production code.
- Ensures the right people review the right changes.
- Helps enforce accountability.

Example:

```
/terraform/   @platform-team
/k8s/         @devops-team
```

6. Merge Requirements

- Require pull request reviews before merging.
- Require a minimum number of approvals.
- Require status checks to pass (tests, security scans).
- Restrict direct pushes to main.
- Use branch protection rules.
- Optionally require signed commits.
- Use merge queues for high-traffic repositories.
- Prevent force pushes and branch deletion.

7. Why Approval Does Not Guarantee a Safe Deployment

- Reviewers can miss issues.
- Tests may not cover all real-world scenarios.
- Infrastructure changes can have unexpected effects.
- External dependencies may fail.
- Runtime configuration or environment issues can cause problems.
- Human error can still happen.
- A successful merge only means the change is good to deploy, not that the deployment will definitely succeed.

Approval reduces risk. Monitoring and verification in production are still essential.

8. Junior Engineer Tip

Before opening a PR, review your own changes with `git diff`. Make sure the PR is small, focused and easy for others to understand.

9. Remember

A pull request is a safety gate, not a deployment.
The real test comes after the change is deployed and verified in production.

Protecting the Main Branch

Keep your production code safe, stable and trustworthy.

1. Why Direct Pushes Can Be Dangerous

- Bypasses code review.
- Introduces untested or unverified changes.
- Can break the production build or deployment.
- Makes it harder to detect who changed what.
- Increases the risk of accidental deletion or force pushes.
- Can introduce security vulnerabilities.
- Reduces accountability and auditability.

Direct pushes to main should be blocked in production environments.

2. Protected Branches

- Protect the main branch using repository settings (for example, GitHub branch protection rules).
- Ensure changes must go through a pull request.
- Define rules for who can push, merge or administer.
- Apply the same protection to other critical branches (if needed).

Protection rules enforce your workflow and reduce risk.

3. Required Reviews

- Require at least one or more approvals before merging.
- Use CODEOWNERS for critical paths (like infrastructure, security or production code).
- Ensure reviewers have the right expertise.
- Prevent self-approval where possible.

More eyes, fewer mistakes.

4. Required Status Checks

- Require CI checks to pass before merging.
- Include build, test, security scans and linting.
- Block merging if any required check fails.
- Use branch protection rules to enforce status checks.

No green checks, no merge.

5. Restricting Force Pushes

- Disable force pushes on main.
- Prevent history rewriting that can remove important commits.
- Keeps the commit history clean, complete and auditable.

Protect your history.

6. Restricting Deletion

- Prevent accidental or intentional deletion of the main branch.
- Keep the branch available for recovery and audit purposes.
- Apply deletion restrictions in repository settings.

The main branch should never be deleted.

7. Signed Commits (Where Appropriate)

- Use GPG or SSH commit signing.
- Helps verify the identity of the author.
- Useful for regulated environments and open-source projects.
- Can be enforced through repository rules.

Signed commits add trust and accountability.

8. Merge Queues

- Merge queues (for example, GitHub merge queue) help manage high-traffic repositories.
- Ensures changes are tested in a queue before merging.
- Reduces merge conflicts.
- Keeps the main branch stable and deployable.

A smoother, safer merge process.

9. Administrator Bypass Risks

- Admins can bypass protection rules.
- Use this power carefully and only in emergencies.
- Maintain an audit log of any bypass.
- Have a clear process for emergency changes.
- Review admin access regularly.

With great access comes greater responsibility.

10. Production Principle

Protect the path through which production changes enter the system.

Junior Engineer Tip

Set up branch protection rules on day one for any production repository. It will save you and your team from serious problems later.

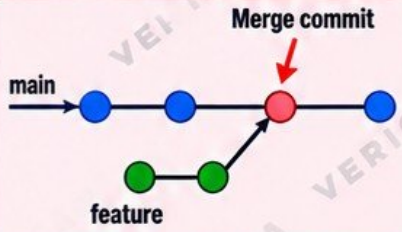
Merge, Rebase and Cherry-Pick in Real Teams

Different tools. Different use cases. Same goal: better, safer production code.

1. What Merge Does

- Combines the changes from one branch into another.
- Creates a new merge commit (with two parents).
- Preserves the full history of both branches.
- Shows exactly when and how the branches were combined.
- Useful for shared branches and long-running development.

Merge creates a new commit

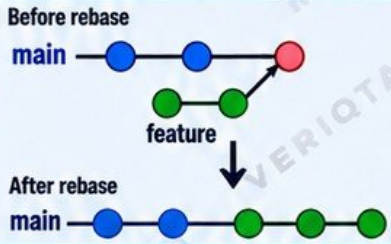


History is preserved.

2. What Rebase Does

- Reapplies your commits on top of another branch.
- Creates a clean, linear history (without merge commits).
- Changes commit hashes (because new commits are created).
- Useful for keeping a clean history before merging (for example, rebasing a feature branch).
- Should not be used on shared branches (for example, main) as it rewrites history.

Rebase creates a linear history

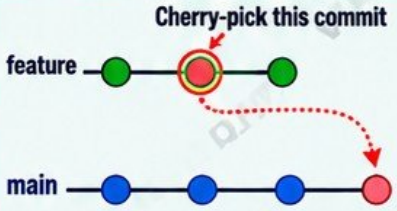


Cleaner, linear history.

3. What Cherry-Pick Does

- Copies a specific commit from one branch to another.
- Creates a new commit with a new hash (different from the original).
- Useful for hotfixes or applying a single change.
- Does not bring the entire branch, only the selected commit.
- Can lead to duplicate-history complexity if the same change exists in multiple places.

Cherry-pick copies a specific commit



Creates a new commit with a new hash.

4. When to Use Each

Scenario	Best Option
Combine feature branch into main (keep history)	Merge
Keep a clean history before merging	Rebase
Apply a single commit (hotfix)	Cherry-pick
Shared branches (main)	Merge (avoid rebase)
Personal feature branch	Rebase (usually safe)
Need one specific change from another branch	Cherry-pick

5. How Each Changes History

Operation	History Impact
Merge	Creates a new commit. Keeps full history (non-linear).
Rebase	Rewrites history. Creates new commit hashes. Linear history.
Cherry-pick	Creates a new commit with a new hash. Does not modify the original branch.

6. Important Things to Remember

- Merging is safer for shared branches because it preserves history.
- Rebasing shared history can cause problems for other team members (who may already have the old commits).
- Cherry-picking is useful for hotfixes but can create duplicate commits and make history harder to follow.
- Choose the right tool based on your team, workflow and deployment process.
- Always communicate with your team before rewriting history.

7. Common Commands

git merge

```
git checkout main
git merge feature
```

Merge a branch into your current branch.

git rebase

```
git checkout feature
git rebase main
```

Reapply your commits on top of another branch.

git cherry-pick

```
git cherry-pick
<commit-hash>
```

Apply a specific commit to your current branch.

Junior Engineer Tip

- Understand your team's branching strategy.
- Use rebase only on your own branches.
- Never rebase a shared branch (for example, main).
- Use cherry-pick carefully and keep track of changes.
- When in doubt, ask a senior engineer.
- Always verify the result with git log, git diff and by running tests.

Production Principle

Use the right Git tool to keep your history clear, your team productive and your production systems safe.

Production Merge Conflicts

Understand. Resolve. Test. Keep moving.

1. Why Conflicts Happen

- Happen when two or more changes modify the same part of a file (or the same file) in different ways.
- Common in active teams, long-running branches and parallel development.
- Can occur during merge, rebase or cherry-pick.
- Not a Git error – it is Git protecting you from losing changes.

2. What Git Can and Cannot Resolve Automatically

Git can often resolve automatically:

- Changes in different lines.
- Changes in different files.
- Simple, non-overlapping changes.

Git cannot resolve automatically:

- Changes to the same lines.
- Conflicting edits to the same code.
- Complex structural changes (like moved or deleted code).
- Logical conflicts (even if the code merges, the logic may still be wrong).

3. Reading Conflict Markers

When a conflict happens, Git adds markers in the file:

```
<<<<<<< HEAD
Your changes here
=========
Their changes here
>>>>>>> branch-name
```

- Everything between `<<<<<<<` and `=======` is your changes (current branch).
- Everything between `=======` and `>>>>>>>` is the incoming changes (merge branch).
- You must edit the file and remove the markers after resolving.

4. Resolving Conflicts Safely

1. Read both changes and understand the intent.
2. Manually edit the file to keep the correct code.
3. Remove the conflict markers.
4. Save the file.
5. Test the changes locally.
6. Stage and continue the merge or rebase.

5. Useful Git Commands

`git status`	Shows files with conflicts and their status.
`git diff`	Shows exactly what changed in the conflicted files.
`git merge --abort`	Cancels the merge and returns to the previous state.
`git rebase --abort`	Cancels the rebase and returns to the previous state.

6. Why "Ours" or "Theirs" Can Be Dangerous

- Using `--ours` or `--theirs` blindly can overwrite important changes.
- You might lose critical bug fixes, security patches or features.
- It can introduce subtle bugs that are hard to detect.
- Always review the changes manually and understand the impact before choosing a side.

Merging is not just about making the conflict go away – it is about keeping the right code.

7. Test After Resolution

- Build the application.
- Run automated tests.
- Test the specific functionality that was changed.
- Check for unexpected side effects.
- For infrastructure code, run plan, validate and test in asafe environment before deploying.

A successful merge is not complete until it is tested.

8. Troubleshooting Flow

Conflict Happens	Understand Both Changes	Resolve	Test	Review Diff	Continue
Git stops and asks you to resolve.	Read your changes and the incoming changes.	Edit the file, keep the correct code and remove conflict markers.	Build, run tests and verify functionality.	Use `git diff` to confirm the changes.	`git add <file>` `git merge --continue` or `git rebase --continue`

Take your time. Understand the changes. Keep your code and production safe.

Production Principle

Resolve conflicts with care. Correct code today prevents incidents tomorrow.

Tags, Releases and Production Versions

Identify. Track. Deploy. With Confidence.

1. Commit IDs

- Each commit has a unique SHA-1 hash (commit ID).
- Example: a3f4e2b1c9d8...
- Identifies the exact changes in the repository.
- Used internally by Git, CI/CD and deployment systems.
- Hard to remember, which is why we use tags and releases.

```
git rev-parse HEAD
# shows the current commit ID
```

2. Tags

- A tag is a human-friendly name that points to a specific commit.
- Marks an important point in the project history.
- Commonly used for releases (e.g. v2.4.1).
- Tags do not change unless someone moves them (which is dangerous).

3. Lightweight vs Annotated Tags

Lightweight Tag

- A simple pointer to a commit.
- No additional metadata.
- Created quickly.
- Useful for temporary or internal tags.

```
git tag v1.0.0
# lightweight tag
```

Annotated Tag

- Stores extra information such as:
  - Tagger name
  - Email
  - Date
  - Message
- Recommended for production releases.

```
git tag -a v1.0.0 -m
"Release v1.0.0"
```

4. Semantic Versioning

vMAJOR.MINOR.PATCH

- MAJOR: Breaking changes (e.g. v2.0.0)
- MINOR: New features (e.g. v2.1.0)
- PATCH: Bug fixes (e.g. v2.1.1)
- Helps teams understand the impact of changes.

5. Release Tags

- Usually follow semantic versioning (e.g. v2.4.1).
- Mark official, stable releases.
- Used by CI/CD pipelines to build and deploy.
- Make it easy to identify the exact code that is running in production.

```
git tag -a v2.4.1 -m "Release v2.4.1"
git push origin v2.4.1
```

6. Why Tags Help Identify Deployed Versions

- Tags provide a clear reference to the exact code that was built, tested and deployed.
- Helps with rollbacks, auditing and troubleshooting.
- Makes it easy to answer: What version is running in production?
- Useful for multiple environments (dev, staging, production).

With tags, you always know what is running, who released it and when.

7. Release Notes

- A summary of changes in a release.
- Helps users, operators and stakeholders understand what is new, fixed or changed.
- Can be stored in the repository (e.g. CHANGELOG.md) or on the GitHub Releases page.
- Should be clear, concise and user-focused.

8. Immutable Release Thinking

- A release should be immutable (unchanged).
- Once a tag is created and released, it should never be modified.
- If a new release is needed, create a new tag (e.g. v2.4.2).
- This ensures consistency, trust and traceability.
- Supports reliable rollbacks and audits.

9. Why Moving or Reusing Tags is Dangerous

- Moving a tag to a different commit rewrites history and breaks trust.
- Can cause confusion about what code is actually running.
- May lead to deploying the wrong code.
- Breaks reproducibility and audit trails.
- Can cause serious issues in production, especially in regulated environments.

Never move or reuse production tags. Always create a new tag.

10. Production Release Flow

Commit → Code changes are committed

→ **Tag** → A tag (e.g. v2.4.1) points to the commit

→ **Build** → CI/CD builds the code

→ **Artifact** → A versioned artifact is created (e.g. Docker image, JAR, Helm chart)

→ **Production Release** → The artifact is deployed to production

Git and CI/CD Pipelines

Automate from code to production.

1. What Triggers a Pipeline?

- A pipeline is triggered automatically when something happens in Git.
- Common triggers:
  - Push events (new commits)
  - Pull requests (open, update, or merge)
  - Tags (for releases)
  - Branch changes
  - Scheduled runs (e.g. nightly builds)
  - Manual trigger (when needed)

Every change should go through an automated pipeline.

2. Push Events

- Triggered when you push commits to a branch.
- Usually runs CI (build, test, security scans).
- Commonly used for feature branches.
- Helps catch issues early before merging.

3. Pull Requests

- Triggered when a pull request is opened, updated or synchronized.
- Runs CI checks (build, test, security scans).
- Helps maintain code quality before merging.
- Can block merging if checks fail.
- Often used with required status checks and reviews.

4. Tags

- Triggered when a tag is created (e.g. v2.4.1).
- Usually used for release pipelines.
- Can trigger both CI and CD.
- Ensures the exact tagged version is built, tested and deployed.
- Helps create consistent and reproducible releases.

5. CI versus CD

CI (Continuous Integration)

- Focuses on building, testing and validating code changes.
- Runs on every push or pull request.
- Helps catch issues early.
- Does not usually deploy to production.

CI = Build, Test, Validate

CD (Continuous Deployment)

- Takes the validated code and deploys it automatically (to staging or production).
- Can be Continuous Deployment (automatic) or Continuous Delivery (manual approval).
- Ensures faster, safer and more consistent releases.

CD = Package, Deploy, Verify

6. Common Pipeline Stages

Build	Compile the code and build artifacts.
Test	Run unit, integration and other tests.
Security Scan	Scan for vulnerabilities (e.g. SAST, DAST).
Package	Create deployable artifacts (e.g. Docker image, JAR, Helm chart).
Deploy	Deploy to staging or production.
Verify	Check application health and functionality.

7. Why Automate with Git?

- Git provides a single source of truth.
- Automation ensures every change is built, tested and deployed consistently.
- Reduces human error compared to manually copying files.
- Provides audit trail and traceability (who changed what, when).
- Enables faster, safer and repeatable deployments.
- Supports infrastructure as code and GitOps.

Production teams let Git trigger automation, not engineers manually copying files.

8. Production Pipeline Flow

Git Push	CI	Test	Build	Artifact	CD	Production
Code is pushed to a branch, PR or tag.	Pipeline is triggered automatically.	Run automated tests.	Build the application and create artifacts.	Store artifacts (e.g. Docker image, JAR, Helm chart).	Deploy automatically (to staging or production).	Application is running in production.

From code commit to production, powered by automation.

9. Production Principle

Every change to production should come from a version-controlled repository and go through an automated pipeline.

10. Junior Engineer Tip

Set up a simple CI pipeline for your project. Even a basic build and test pipeline will teach you a lot.

11. Remember

No manual file copying to production. If it is not in Git and not in the pipeline, it does not exist.

Git, Artifacts and Traceability

From code to production. Know what is running, why it is running, and how to reproduce it.

1. Source Code is Not the Deployed Artifact

- Source code is human-readable and can change.
- It must be built, tested and packaged before deployment.
- What runs in production is usually a built artifact (e.g. container image, JAR, RPM).
- Never deploy source code directly to production.

Code in Git is the starting point, not the final product.

2. Build Artifacts

- Build artifacts are the output of the build process.
- Examples: JAR files, WAR files, ZIP files, binary packages.
- Stored in artifact repositories (e.g. JFrog Artifactory).
- Each artifact is linked to a specific commit, build number and version.

Artifacts are immutable evidence of what was built.

3. Container Images

- Container images package your application and its dependencies.
- Built from source code (e.g. Dockerfile).
- Stored in container registries (e.g. Docker Hub, ECR, Artifactory).
- Identified by a tag (e.g. v2.4.1) and a digest (immutable).

Use image digests in production for true immutability.

4. Package Repositories

- Package repositories store reusable components (libraries, dependencies).
- Examples: Maven, npm, PyPI, NuGet, RPM repositories.
- Used during the build process to fetch dependencies.
- Should be versioned and scanned for security.

They help ensure consistent, repeatable builds.

5. Key Identifiers

Commit SHA	Unique hash that identifies the exact source code.
Release Version	Human-friendly version (e.g. v2.4.1).
Image Digest	Immutable identifier for a container image (e.g. sha256:...).
Artifact Version	Version of the built artifact (e.g. app-2.4.1.jar).

All identifiers should be linked and traceable across the pipeline.

6. Connecting Production to Source

- A production workload should show the exact version, image digest and commit SHA.
- Use labels/annotations in container images (e.g. org.opencontainers image.revision).
- Store metadata in your deployment system (e.g. Kubernetes, Helm).
- Make it easy to trace from production back to Git, build and release information.

You should be able to answer "what code is running?" in seconds.

7. Why Reproducibility Matters

- Allows you to recreate the exact same artifact in the future.
- Helps with incident investigation and debugging.
- Ensures consistency across environments.
- Required for compliance, auditing and security.
- Reduces "works on my machine" problems.

If you can't reproduce it, you can't fully trust it.

8. Real World Question

"Which exact commit is currently running in production?"

- This question should be easy to answer.
- The production system should make this information discoverable.
- Check labels, deployment metadata, monitoring dashboards or a simple API.
- If it is hard to find, your traceability is not good enough.

Production without traceability is a risk.

9. End-to-End Traceability Flow

Commit	Code is committed to Git (commit SHA).
↓ Build	CI builds the code and creates artifacts.
↓ Artifact	Artifact is stored (e.g. JAR, image).
↓ Deploy	Artifact is deployed to production.
↓ Production	Production system shows version, digest and commit SHA.

From commit to production, every step should be traceable.

10. Production Principle

Make the link between code, artifacts and production visible and easily discoverable.

Junior Engineer Tip

Add labels with the commit SHA and version to your container images and deployments.

11. Remember

Source code changes, but tags, artifacts and metadata help you know exactly what is running.

Git for Infrastructure as Code

Version control for a reliable, repeatable and auditable infrastructure.

1. Terraform in Git

- Store all Terraform code in Git repositories.
- Use modules, clear structure and naming.
- Track changes through commits and pull requests.
- Review terraform plan before applying.
- Keep environments (dev, staging, prod) in separate folders or repos if needed.

```
terraform/
├── modules/
├── environments/
├── main.tf
└── variables.tf
```

2. Ansible in Git

- Store playbooks, roles, inventories and configuration in Git.
- Track all changes and use pull requests.
- Review changes before running in production.
- Use clear directory structure and documentation.
- Integrate with CI/CD for validation (syntax checks, ansible-lint).

```
ansible/
├── playbooks/
├── roles/
├── inventories/
├── group_vars/
└── host_vars/
```

3. Cloud Configuration in Git

aws

- Store cloud configuration as code (e.g. Terraform, CloudFormation, Pulumi or native tools).
- Keep all infrastructure settings, policies and templates in Git.
- Track changes, use pull requests and review before deployment.
- Manage multi-environment configurations.
- Document architecture and changes in the repository.

Cloud infrastructure should be versioned, reviewed and auditable just like application code.

4. Pull-Request-Driven Infrastructure Changes

- All infrastructure changes should go through a pull request (PR).
- Enables team review, discussion and automatic checks.
- Helps catch misconfigurations before they reach production.
- Links infrastructure changes to an issue or task.
- Creates a clear audit trail of who changed what and why.

No direct changes to production infrastructure.

5. terraform plan During Review

- Run terraform plan in CI during the pull request.
- Shows exactly what will be created, changed or destroyed.
- Helps reviewers understand the impact before approval.
- Catches unexpected changes early.
- Reduces the risk of accidental destruction.

```
terraform plan
```

Review the plan output carefully. If it is not expected, do not approve.

6. Applying Approved Changes

- Only apply changes after the PR is approved and merged.
- Use CI/CD to run terraform apply or ansible-playbook.
- Apply in a controlled and consistent manner (e.g. automated pipeline).
- Use proper approvals for production.
- Monitor the apply process and verify the result.

Approved code → Automated apply → Verified infrastructure.

7. State Files and Why They Require Special Handling

- Terraform state files contain the current state of your infrastructure.
- They include sensitive information (e.g. resource IDs, metadata).
- Store state files in a remote backend (e.g. S3, Azure Storage, GCS) with proper access controls.
- Do not commit state files to Git.
- Use state locking to prevent concurrent changes.

State files are critical. Protect them.

8. Why Credentials Must Not Be Committed

- Cloud credentials, API keys, tokens and secrets can be abused if exposed.
- Never commit credentials to Git.
- Use secret management tools (e.g. AWS Secrets Manager, Azure Key Vault, HashiCorp Vault).
- Use environment variables or CI/CD secrets.
- Scan repositories for secrets (e.g. git-secrets, truffleHog).

If it is a secret, keep it out of Git.

9. Infrastructure Change History

- Git provides a complete history of all infrastructure changes.
- You can see who changed what, when and why.
- Helps with auditing, compliance and troubleshooting.
- Makes it easy to roll back if needed.
- Links changes to issues, PRs and approvals.

Your infrastructure history is a valuable record. Keep it clean and accurate.

10. Infrastructure Change Flow

Terraform Change	→ PR	→ Plan	→ Review	→ Merge	→ Controlled Apply
Make changes to Terraform code in Git.	Open a pull request with the changes.	Run terraform plan during review.	Team reviews the plan and approves.	Merge the PR into main branch.	CI/CD runs terraform apply in a controlled manner.

Production Principle

Treat infrastructure code with the same discipline as application code.

GitOps and Kubernetes

Declarative. Automated. Always in Sync.

1. What GitOps Means

- GitOps is a way of managing infrastructure and applications using Git as the single source of truth.
- Changes are made in Git and automatically applied to the cluster by a GitOps controller.
- Everything is versioned, auditable and reproducible.
- Removing manual changes reduces errors and ensures consistency.

GitOps = Infrastructure and applications as code, stored in Git, continuously reconciled to the cluster.

2. Desired State

- Desired state is the configuration you want the cluster to be in (e.g. Kubernetes manifests and Helm values).
- You define the desired state in Git.
- The GitOps controller makes sure the actual state matches the desired state.
- If there is a difference, the controller automatically reconciles it.

**You describe what you want, not how to do it.
This is declarative infrastructure.**

3. Kubernetes Manifests

- Kubernetes manifests are YAML files that define resources (e.g. Deployment, Service, Ingress, ConfigMap).
- Stored in Git and version controlled.
- Can be applied manually or through a GitOps controller.
- Helps ensure consistent, repeatable deployments across environments.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
```

4. Helm Values

- Helm values files define configuration for Helm charts.
- Stored in Git and managed like any other code.
- Environment-specific values (e.g. dev, staging, prod). Make it easy to customize applications without changing the chart.
- Changes to values in Git are automatically applied by the GitOps controller.

Helm + GitOps = Consistent, version-controlled deployments.

5. Git as Desired-State Source

- Git stores the desired state (Kubernetes manifests, Helm values, and related configuration).
- Every change is tracked with history, approvals and pull requests.
- Acts as the single source of truth for both infrastructure and applications.
- Allows collaboration and rollback if needed.

If it is not in Git, it does not exist (in GitOps).

6. GitOps Controller

- A GitOps controller (e.g. Argo CD, Flux) monitors the Git repository for changes.
- It automatically applies the desired state to the Kubernetes cluster.
- It continuously checks for drift and reconciles differences.
- Provides visibility, sync status and history of deployments.

**Popular GitOps controllers:
Argo CD | Flux CD**

7. Reconciliation

- The controller continuously compares the desired state in Git with the actual state in the cluster.
- If there is a difference, it automatically applies the changes to bring the cluster back to the desired state.
- This process is called reconciliation.

Reconciliation ensures the cluster is always in the state defined in Git.

8. Drift

- Drift happens when the actual cluster state is different from the desired state in Git.
- Can be caused by manual changes in the cluster (e.g. kubectl edit), failed updates or external tools.
- The GitOps controller detects drift and automatically corrects it during reconciliation.

Drift is why you should not make manual changes in the cluster.

9. Why Manual Changes are a Problem

- Manual changes (e.g. kubectl edit, kubectl apply) can create configuration drift.
- The changes are not stored in Git, so they are not version controlled.
- They can be overwritten by the GitOps controller.
- They make it harder to track who changed what and why.

Always make changes through Git, not directly in the cluster.

10. GitOps Flow

Engineer	Git	GitOps Controller	Kubernetes Cluster
Makes changes in Git (manifests or Helm values).	Stores the desired state (version controlled).	Detects changes and reconciles the cluster.	Runs the desired state (automatically kept in sync).

11. Production Principle

Keep your infrastructure and applications declarative, version controlled and managed through GitOps. Avoid manual changes.

**Git is the source of truth.
The cluster follows Git.**

Secrets and Sensitive Data in Git

Protect your secrets. Protect your systems. Protect your career.

1. What Are Secrets and Sensitive Data?

- Passwords
- API keys
- SSH private keys
- Cloud credentials (AWS, Azure, GCP)
- Tokens (OAuth, GitHub tokens, etc.)
- .env files
- Database connection strings
- Certificates and private keys
- Any information that can be used to gain unauthorized access

If it can give access, it is a secret.

2. Why .gitignore is Preventive, Not a Cleanup Tool

- .gitignore tells Git which files to ignore in the future.
- It does not remove files that are already committed.
- If a secret has already been committed, it will remain in Git history even if you add it to .gitignore later.
- Use .gitignore from the start, especially for .env files, key files and credential files.

.gitignore prevents new mistakes. It does not fix old ones.

3. Git History Retains Committed Secrets

- Every commit is stored in the repository history.
- Even if you delete a file, the secret may still exist in previous commits.
- Anyone with access to the repo can potentially retrieve it.
- Secrets in history can be found using `git log`, `git show` or automated secret scanning tools.

If you committed a secret, assume it is exposed.

4. Secret Scanning

- Platforms like GitHub, GitLab and Bitbucket can automatically scan for secrets.
- They detect patterns such as API keys, tokens and credentials.
- Use pre-commit hooks and CI/CD checks to prevent accidental commits.
- Regularly scan your repositories for exposed secrets.

Automated scanning helps you catch mistakes early.

5. Credential Rotation

- If a secret has been committed, assume it is compromised.
- Immediately rotate or revoke the credential (new password, new key, new token).
- Update all systems and environments using the old credential.
- Monitor for any suspicious activity after rotation.

Rotate first, investigate next.

6. Removing Sensitive History

- Use tools like git filter-repo or BFG Repo-Cleaner to remove secrets from the entire history.
- This rewrites the repository history.
- Force push the cleaned history (coordinate with your team).
- Also remove forks if the repo is public (they may still contain the old history).

Removing secrets from history is a serious action. Plan and communicate.

7. Secrets Managers

- Use dedicated secrets managers:
- AWS Secrets Manager
- Azure Key Vault
- Google Secret Manager
- HashiCorp Vault
- Store and access secrets securely at runtime, not in Git.
- Use environment variables or workload identity to access secrets.

Secrets belong in a secrets manager, not in your Git repository.

8. What to Do Immediately After Accidentally Committing a Credential

1. Revoke or rotate the credential immediately.
2. Assess the exposure (public repo? internal repo? who has access?).
3. Remove the secret from the repository history.
4. Check for any unauthorized usage.
5. Update your processes to prevent recurrence.

Act quickly. Assume it is already exposed.

9. Production Best Practices

- Never commit secrets, keys or .env files.
- Use .gitignore from the start.
- Use secrets managers.
- Enable secret scanning.
- Rotate credentials regularly.
- Follow least privilege access.
- Educate your team.
- Have an incident response plan.

Security is a continuous habit, not a one-time fix.

10. Incident Flow: Accidental Secret Commit

Secret Committed	Revoke / Rotate	Assess Exposure	Remove From Repository / History	Investigate	Prevent Recurrence
A credential is committed to Git.	Immediately revoke or rotate the credential.	Check who has access and if it was public.	Use git filter-repo or BFG and force push.	Look for any unauthorized usage.	Update processes, add scanning and educate the team.

Undoing Changes Safely

Different commands. Different purposes. Use the right one.

1. git restore (Discard Local Changes)

- Used to discard changes in your working directory or staging area.
- Does not affect commit history.
- Useful when you changed a file but do not want to keep the changes.
- Safe for local, uncommitted changes.

Example:

```
git restore file.txt
git restore --staged file.txt
```

What it does:

Reverts the file to the last committed version.

2. git revert (Create a New Commit)

- Creates a new commit that undoes a previous commit.
- Does not rewrite history.
- Safe to use on shared branches (including main).
- Commonly used to revert production changes.
- Keeps a full audit trail.

Example:

```
git revert <commit-hash>
```

What it does:

Creates a new commit that reverses the changes made in the specified commit.

3. git reset (Move the Branch Pointer)

- Moves the current branch to a different commit.
- Can keep changes, unstage them or delete them.
- Commonly used for local history cleanup.
- Be very careful on shared branches because it rewrites history.

Examples:

```
git reset --soft <commit-hash>
git reset --mixed <commit-hash>
git reset --hard <commit-hash>
```

What it does:

Moves the branch pointer and changes how your working files are handled.

4. Comparing the Commands

Command	Affects History?	Typical Use Case	Safe on Shared Branch?
git restore	No	Discard local changes in working directory or staging area	Yes
git revert	No (creates new commit)	Undo a commit in a safe and auditable way	Yes
git reset	Yes (moves branch)	Move to a previous commit (clean up local history)	No (if already pushed)

5. Reverting a Production Change

- Use git revert to undo the bad commit.
- Test the change in a non-production environment if possible.
- Create a new pull request and follow the normal review process.
- Deploy the revert through your CI/CD pipeline.
- Verify the application is working.
- Document the reason for the revert in the commit message.

6. Local vs Shared History

- Local history exists only on your machine.
- You can freely use git reset, rebase or other history changes locally.
- Shared history is on a remote repository (for example GitHub).
- Rewriting shared history can break other people's work.
- Always be careful with history changes after pushing to a shared branch.

7. Why reset --hard Can Destroy Work

- **git reset --hard** moves the branch and deletes your local changes.
- All uncommitted work in the working directory is lost.
- There is no easy way to recover it unless it exists in the reflog.
- Use with extreme caution.
- Make sure you really want to discard those changes.

8. Why Rewriting Shared History Is Risky

- Other team members may have already pulled the commits.
- It can cause merge conflicts.
- It can break open pull requests.
- It can make it difficult to track what actually happened.
- It reduces transparency and auditability.
- Use force pushes only when you fully understand the impact.

9. Revert Commits as Auditable Recovery

- A revert creates a new commit, so the history remains intact.
- It shows exactly what was changed and when it was reverted.
- It is the preferred method for undoing changes in production.
- It helps with compliance, auditing and incident reviews.

10. Production Rule

Prefer a traceable reversal over casually rewriting shared history.

Keep your history clear, auditable and safe for your team.

Remember This

Git is powerful.
Use the right command, understand the impact, and think about your team.

Production Rollbacks

Recover safely. Learn from the incident. Deploy with confidence.

1. Git Revert vs Application Rollback

- git revert: creates a new commit that undoes a previous change in the repository.
- Application rollback: redeploys a previous working version of the application (for example, a container image or package).
- These are not the same operation.
- You can revert code without redeploying, and you can redeploy without reverting code.
- Both may be needed during an incident and recovery.

2. Why They Are Not the Same

- git revert changes the source code history.
- Application rollback changes what is running in production.
- You might need to rollback a deployment quickly, then revert the code intentionally and properly.
- Sometimes you roll back the application but keep the code change and fix it.
- Sometimes you revert code but do not need to deploy immediately.

3. Rolling Back Deployments

- Use your deployment tool (for example, Kubernetes, ECS, or your CD system).
- Redeploy the last known-good artifact.
- Do not rebuild from the same commit again.
- Use versioned artifacts (images, packages).
- Verify that the previous version is healthy.

4. Common Considerations

Database Migrations	Rolling back application code may not roll back database changes. Plan ahead.
Backward Compatibility	Ensure the previous version is compatible with current data and services.
Feature Flags	Use feature flags to quickly disable new features without a full rollback.

5. Roll-Forward vs Rollback

Rollback	Roll-Forward
Go back to a previous known-good version.	Fix the issue and release a new version.
Useful for quick recovery.	Often safer in complex systems.
May not always be possible (for example, database changes).	Recommended when rollback is risky or not possible.
	Learns from the incident and improves the system.

6. Typical Rollback Flow

Bad Release v2.5	A new release causes issues in production.
↓	
Incident	Errors, degraded performance or outage.
↓	
Rollback to Known-Good Artifact	Redeploy the last known-good version.
↓	
Verify Service	Confirm the service is healthy and stable.
↓	
Fix Source	Identify the root cause and fix the code.
↓	
New Reviewed Release	Deploy a new, reviewed and tested release.

7. Verification After Recovery

- Check application health (logs, metrics, dashboards).
- Verify key user journeys and functionality.
- Monitor for a period of time to ensure stability.
- Communicate with the team and stakeholders.
- Document what happened and the actions taken.

Production Rule

Prefer a traceable reversal over casually rewriting shared history.

Use git revert or a controlled rollback process so changes are auditable and the team can learn from the incident.

Remember This

A fast rollback fixes the symptom.
A proper fix prevents it from happening again.

Git During a Production Incident

Stay calm. Investigate. Identify the change. Fix safely.

1. Do Not Panic-Edit Production

- Do not make random changes in production.
- Understand what changed and why.
- Use Git history, deployment history and logs.
- Follow a structured troubleshooting process.
- Communicate with your team.
- Make minimal, targeted changes.

2. Determine What Changed

- Check recent deployments.
- Look at recent commits.
- Compare the last known-good release with the failing release.
- Identify who made the change, when and what was changed.
- Use Git commands and your CI/CD or deployment tool.
- Check if the issue matches a specific commit or change.

3. Useful Git Commands

`git log`	Shows commit history.
`git show <SHA>`	Shows details of a specific commit.
`git diff <SHA1> <SHA2>`	Compares changes between commits.
`git diff main`	Shows changes in your working directory.
`git blame <file>`	Shows who changed each line and when.

4. Commit SHA Comparison

Compare the last known-good commit with the failing commit.

```
git diff <good-sha> <bad-sha>
```

This shows exactly what changed between the two versions.

5. Find the Suspected Change

- Use git log to review recent commits.
- Look for changes related to the issue (for example, configuration, code, Docker image, or infrastructure).
- Use git show <SHA> to see details.
- Check pull requests and commit messages.
- Correlate with deployment time, logs and monitoring alerts.

6. Reverting Safely

- Use git revert <SHA> to create a new commit that undoes the change.
- This keeps history intact and is safer for shared branches.
- Test the revert in a non-production environment if possible.
- Deploy through your normal CI/CD pipeline.
- Verify the service after the rollback.

7. Emergency or Hotfix Workflows

- For critical incidents, a hotfix branch may be used.
- Make the smallest possible change to fix the issue.
- Follow the same review and deployment process.
- Clearly mark the commit as an emergency fix (for example, in the commit message).
- After the incident, merge the hotfix back to main to keep history consistent.

8. Record After the Incident

- Document what happened.
- Record the timeline (when the change was made, by whom, and the impact).
- Link relevant commits, pull requests and incident tickets.
- Write a clear postmortem.
- Update runbooks to prevent recurrence.
- Share the learnings with the team.

9. Preserve Evidence

- Do not delete or rewrite history.
- Keep logs, commit SHAs, deployment records and chat messages.
- This helps with root cause analysis, auditing and future improvements.
- Treat the incident as a learning opportunity, not a blame game.

10. Typical Investigation Flow

Identify the Issue — Service is failing or behaving unexpectedly.
→ **Check Deployment History** — Find the last deployment time and version.
→ **Review Git Changes** — Use git log, git diff and git show.
→ **Find the Suspected Change** — Compare with last known-good version.
→ **Revert or Fix** — Use git revert or hotfix.
→ **Verify Service** — Confirm the environment is healthy.
→ **Document** — Record actions and write postmortem.

Troubleshooting Question

"What changed between the last known-good release and the failing release?"

Production Awareness

- Use Git to find the change, not to guess.
- Keep history, communicate clearly and learn from the incident.

Junior Engineer Tip

In a production incident, clarity is more important than speed.

Auditing and Troubleshooting Git History

Use Git history to understand, debug and find the truth.

1. Key Git History Commands

`git log`	Shows commit history.
`git show`	Shows details of a specific commit.
`git diff`	Compares changes between commits, branches or files.
`git blame`	Shows who changed each line and when.
`git reflog`	Shows a local record of recent HEAD movements (even deleted commits).

2. What Each Command Tells You

- `git log` Which commits were made, by whom and when.
- `git show` The exact changes in a commit (files, additions, deletions).
- `git diff` What changed between commits, branches, tags or your working directory.
- `git blame` Which commit last modified each line and who made the change.
- `git reflog` A history of where your HEAD has been (useful for recovery).

3. Useful Examples

`git log --oneline --graph`
Compact visual history.

`git show <commit-sha>`
Show details of a commit.

`git diff <sha1> <sha2>`
Compare two commits.

`git diff main` See changes from your current branch to main.

`git blame <file>`
See who changed each line.

`git reflog` View recent HEAD history.

4. Local Reflog Limitations

- git reflog exists only on your local machine.
- It tracks local actions (commits, checkouts, resets), not a full remote history.
- Entries can expire and be garbage collected over time.
- It is not available to other team members.
- It is useful for recovering "lost" commits locally, but not a backup strategy.

⚠ Reflog is powerful but temporary. Do not rely on it as long-term history.

5. Finding When Behavior Changed

- Use git log to find recent changes around the time the issue started.
- Use git diff to compare working versions.
- Use git blame to identify the exact line and commit.
- Check commit messages for context (jira tickets, issue numbers).
- Look at deployment history to match the commit with the production release.
- Review related files (configuration, Dockerfile, manifests, etc.).

6. Comparing Releases

- Use git diff <tag1> <tag2> to see what changed between releases.
- Compare branches (for example main vs release).
- Review changelogs or release notes if available.
- Check specific files or folders (for example configuration).
- Focus on the commits, not just the file list.
- This helps you understand why a new version behaves differently.

7. Investigating Deleted or Moved Work

- Use git reflog to find commits that were reset or deleted (locally).
- Use git log --all to search all branches.
- Use git show <commit-sha> to inspect a commit.
- Look for stale branches, tags or pull requests.
- Check your remote repository (GitHub, GitLab, etc.) for closed pull requests.
- You can often recover lost work if the commit still exists in reflog or on the remote.

8. Why git blame Identifies History, Not Human Blame

- git blame shows who last modified each line, but it does not tell you why.
- Code changes are often made by many people over time.
- The person who wrote the code might not be the person who introduced a bug.
- Use blame to understand context, not to assign fault.
- Focus on solving the problem together.

Practice Lab

Find the commit that introduced a configuration change.

1. Identify a configuration file (for example, app.yaml, nginx.conf, or a .env file).
2. Use git log and git blame to find the commit that changed a specific setting.
3. Use git show to review the full change.
4. Compare with the previous version using git diff.
5. Write a short summary of what changed, when and by whom.

Production Tip

When troubleshooting, use Git to find facts, not assumptions.

Remember This

Git history is a powerful debugging tool. It helps you understand what changed, when and why.

Junior Engineer Tip

Learn these commands and practice them regularly. They will save you time during real incidents.

Production Git Workflow Final Practical Lab

Same Engineers Different Results

From Change to Production. Put Everything Together.

Practice
Build
Deploy
Troubleshoot
Grow

1. Scenario

You are a junior DevOps engineer responsible for changing an application configuration used in production.

Your goal is to make the change safely, go through the full Git workflow, and handle a simulated failure.

2. Step-by-Step Lab

1. Pull the latest repository state.
2. Create a short-lived branch.
3. Make the configuration change.
4. Inspect `git diff`.
5. Commit it clearly.
6. Push the branch.
7. Open a pull request.
8. Let CI validate it.
9. Review and approve the change.
10. Merge through the protected workflow.
11. Build an immutable artifact.
12. Deploy through CD.
13. Verify production health.
14. Detect a simulated failure.
15. Identify the deployed commit.
16. Recover using the approved rollback strategy.
17. Verify recovery.
18. Document what happened.

3. Final Architecture

Engineer
↓
Feature Branch
↓
Pull Request
↓
Review + CI + Security Checks
↓
Protected Main
↓
Build
↓
Versioned Artifact
↓
Deployment
↓
Production
↓
Monitoring
↓
Verify or Roll Back

4. Key Concepts to Apply

- Use a feature branch, not direct pushes.
- Write clear, meaningful commits.
- Use pull requests and code reviews.
- Let automated CI tests validate changes.
- Deploy versioned, immutable artifacts.
- Verify the application in production.
- Know how to identify and roll back if needed.
- Document everything for learning and postmortems.

5. Useful Git Commands

`git clone`	Get the latest code.
`git switch -c <branch>`	Create a new branch.
`git diff`	See your changes.
`git commit -m "message"`	Commit changes.
`git push origin <branch>`	Push to remote.
`git log --oneline`	View commit history.
`git show <sha>`	See commit details.
`git diff <sha1> <sha2>`	Compare commits.
`git revert <sha>`	Create a rollback commit.

6. Verification Checklist

- [x] Deployment completed successfully.
- [x] Application is healthy (logs, metrics).
- [x] Functionality works as expected.
- [x] Correct version is running.
- [x] Monitoring shows no errors.
- [x] Rollback process tested and working.

7. Troubleshooting Questions

- What changed between the last known-good release and the failing release?
- Which commit is currently deployed?
- Are there any error messages in the logs?
- Did the configuration change apply correctly?
- Do we need to roll back or roll forward?

Production Tips

- Take small, incremental changes.
- Always review the diff before committing.
- Never commit secrets or sensitive data.
- Use clear commit messages.
- Follow your team's branching and release strategy.
- Verify in a non-production environment when possible.
- Learn from every incident.

9. What You Learn From This Lab

- How Git fits into real production workflows.
- How to make safe, controlled changes.
- How to handle failures and rollbacks.
- How to trace changes in production.
- The importance of automation, reviews and monitoring.
- How to document and communicate.
- How to be a more confident, production-ready engineer.

10. Final Reminder

Good engineers write code.
Great engineers operate it safely in production.

Learn. Practice. Deploy. Grow.

Consistent Learning Real Progress

Instagram @veriqta | GitHub @veriqta | LinkedIn @veriqta | Telegram @veriqta | Medium @veriqta | X @veriqta

Build Operate Improve

SAME ENGINEERS DIFFERENT RESULTS